Project Interim Report
On

File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming With Java

SESSION: 2025-26

Submitted By
ABHISHEK KUMAR KASHI
ERP no.-RU-25-10056

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with GOOGLE

Under the Guidance of
Mr. Shivansh Mehta

SCHOOL OF COMPUTER SCIENCE AND E NGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

TABLE OF CONTENT

SERIAL NO.	TITLE	PAGE NO.
1.	ABSTRACT	
2.	INTRODUCTION	
3.	SCOPE	
4.	SYSTEM REQUIREMENT	
5.	METHODOLOGY	
6.	FLOWCHART	
7.	IMPLEMENTATION	
8.	GANTT CHART	
9.	CONCLUSION	
10.	REFERENCE	

Abstract

The Online Auction System is a console-based Java application designed using Object-Oriented Programming (OOP) concepts. The system allows multiple users (bidders) to participate in auctions and place bids on different items within a specified time.

The main purpose of this project is to simulate a real-world auction environment where users can view items, place bids, and determine winners based on the highest bid. It ensures that only valid bids (higher than the current highest bid) are accepted and prevents bidding once the auction ends.

This project demonstrates the use of classes, objects, methods, and collections like ArrayList. It also highlights validation logic and system control through menu-driven interaction. The system is useful for understanding real-time bidding logic and applying OOP principles in Java programming.

INTRODUCTION

The Online Auction System is a software application that enables users to participate in bidding for various items. Auctions are commonly used in e-commerce platforms where buyers compete by placing higher bids.

This project is developed using Java and follows Object-Oriented Programming principles. It includes features such as:

- Viewing available auction items
- Starting auctions
- Placing bids
- Validating bid amounts
- Declaring winners

The system ensures fairness by allowing only higher bids and restricting actions after the auction ends. It provides a simple console interface for interaction.

OBJECTIVES

- Simulate a real-world auction system
- Manage auction items and bidders
- Validate bids effectively
- Track highest bid and bidder
- Apply Object-Oriented Programming concepts

Scope

This project is suitable for small-scale auction systems where users can place bids on items in a controlled environment. It can be used for learning and demonstration purposes to understand how real-world auction platforms work.

The system can be further extended in the future by:

- Integrating a database for storing items and bidder details
- Developing a web-based or mobile application interface
- Adding real-time auction timers and notifications
- Supporting multiple users simultaneously over a network
- Maintaining complete bid history and analytics

System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Processor: Intel i3 or above
- Hard Disk: At least 500 MB free space

Software Requirements

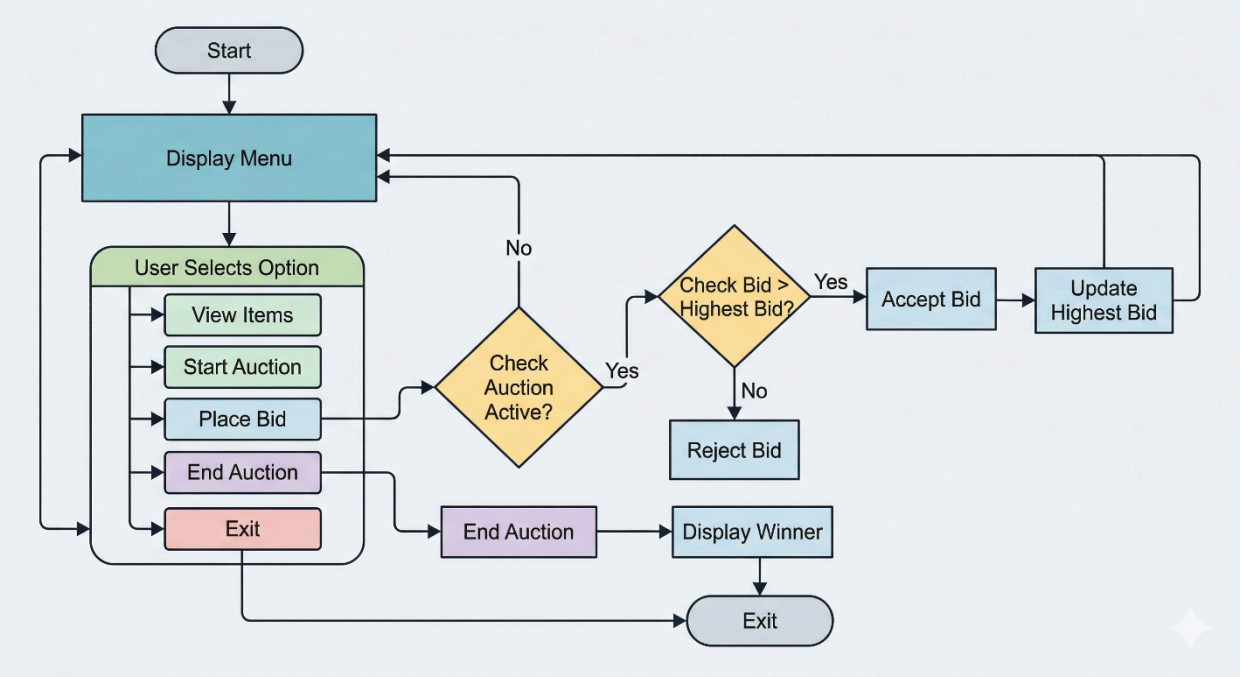
- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK)
- IDE: VS Code / Eclipse / IntelliJ IDEA / Edit Plus
- Java Runtime Environment (JRE)

Methodology

The system works in the following steps:

1. Initialize auction items and bidders
2. Display menu options to the user
3. User selects an operation:
    - View items
    - Start auction
    - Place bid
    - End auction
4. Validate user inputs
5. Update bid details if valid
6. Display auction results

Flowchart



1. Class: Item

Variables inside class

- private String name;
- private double price;

Methods inside class

- public Item(String name, double price) → Constructor
- public String getName() → returns item name
- public double getPrice() → returns item price

2. Class: Auction

Variables inside class

- private Item[] items; (Array present)
- private boolean auctionActive;
- private int currentItemIndex;
- private double highestBid;
- private String highestBidder;

Methods inside class

- public Auction(Item[] items) → Constructor
- public void viewItems() → display all items
- public void startAuction(int index) → start auction for selected item
- public void placeBid(String bidderName, double bidAmount) → place bid
- public void endAuction() → end auction
- public void displayWinner() → show winner

Implementation

Item Class:

The Item class is the core of the system. It stores all auction-related data for a single item including itemId, itemName, basePrice, highestBid, highestBidder, and a boolean flag auctionActive. The startAuction() method sets the auction status to active. The isAuctionActive() method returns whether the auction is currently running. The updateBid() method checks if the incoming bid is greater than the current highest bid and if the auction is active — if both conditions are true, it updates the highest bid and bidder name; otherwise, it rejects the bid. The endAuction() method sets the auction as inactive and prints the winner details.

Bidder Class:

The Bidder class represents a participant in the auction. It has two attributes: bidderId and bidderName. The placeBid() method takes an Item object and a bid amount as parameters and calls the item's updateBid() method, passing the bidder's name and bid amount.

OnlineAuctionSystem Class (Main):

This is the main driver class. It initializes two ArrayLists — one for items and one for bidders — with pre-loaded sample data. A do-while loop runs the main menu continuously until the user chooses to exit. The viewItems() method prints all items and their current status. The startAuction() method lets the user select an item by ID and start its auction. The placeBid() method lets the user choose a bidder and an item, then enter a bid amount which is validated. The endAuction() method ends the auction and shows the result. Two helper methods findItem() and findBidder() search the ArrayLists by ID and return the matching object.

Gantt Chart

Task	Start Day	End Day	Duration
Planning	Day 1	Day 1	1 Day
System Design	Day 2	Day 3	2 Days
Coding	Day 4	Day 6	3 Days
Testing	Day 7	Day 7	1 Day
Documentation	Day 8	Day 8	1 Day

Conclusion

The Online Auction System project successfully demonstrates the practical application of Object-Oriented Programming concepts using Core Java. Through the design and implementation of the Item, Bidder, and main system classes, the project shows how real-world business logic can be modeled using encapsulation, class design, and data validation.

The system fulfills all functional requirements including viewing items, placing bids, validating bid amounts, tracking the highest bidder, and announcing the winner after the auction ends. The use of ArrayList for data storage makes the system flexible and easy to extend.

This project helped in understanding how classes interact with each other, how methods communicate data, and how to design a menu-driven console application in Java. It provides a strong foundation for building more advanced systems with database integration and graphical interfaces in the future.

References

Books:

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.

[2] E. Balagurusamy, Programming with Java: A Primer, 5th ed. New Delhi: McGraw-Hill, 2014.

Websites:

[3] Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com

[4] GeeksforGeeks, "Object Oriented Programming in Java," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/

Journal Articles:

[5] A. Sharma and R. Gupta, "Object Oriented Design Patterns in Java Applications," International Journal of Computer Science, vol. 5, no. 2, pp. 45-50, 2020.